TÀI LIỆU CHUYÊN KHẢO KỸ THUẬT PHẦN MỀM

Kiến trúc SOA & Microservices

Đặng Ngọc Hùng

Mục lục

11. Quan sát hệ thống — Observability & Distributed Tracing . . . 2
11.1. Ba trụ cột của observability . . . 3
11.1.1. Vấn đề: "hệ thống lỗi nhưng không biết lỗi ở đâu" . . . 3
11.1.2. Ba trụ cột . . . 3
11.1.3. Observability và monitoring . . . 4
11.1.4. Mô hình trưởng thành observability (observability maturity model) . . . 5
11.2. Logging — từ console đến gom log tập trung . . . 5
11.2.1. Vấn đề: log phân tán, không có ngữ cảnh . . . 6
11.2.2. Log có cấu trúc — từ văn bản thô sang JSON . . . 6
11.2.3. Gom log tập trung (centralized log aggregation) . . . 7
11.2.4. Định danh xuyên suốt: Request ID và Correlation ID . . . 7
11.3. Chi tiết về Distributed Tracing . . . 9
11.3.1. Vấn đề: request chậm — chậm ở đâu? . . . 9
11.3.2. Distributed Tracing — tự động đo thời gian . . . 9
11.3.3. Các khái niệm: Trace, Span, Context Propagation . . . 10
11.3.4. Chiến lược lấy mẫu (Sampling) . . . 10
11.3.5. Tracing trong Spring Boot 3.x . . . 11
11.3.6. Kiến trúc triển khai Observability Pipeline . . . 12
11.4. Metrics, SLI/SLO và chiến lược cảnh báo . . . 13
11.4.1. Metrics — đo lường hành vi theo thời gian . . . 13
11.4.2. Bốn loại metric cơ bản của Prometheus . . . 13
11.4.3. Hai khung đo lường: RED và USE . . . 14
11.4.4. SLI/SLO/SLA — từ metric đến cam kết chất lượng . . . 14
11.4.5. Chiến lược cảnh báo . . . 15
11.5. Chiến lược xử lý lỗi — nhất quán giữa các service . . . 16
11.5.1. Vấn đề: mỗi service trả về định dạng lỗi khác nhau . . . 16
11.5.2. Nguyên tắc thiết kế xử lý lỗi . . . 16
11.5.3. Mối liên hệ giữa xử lý lỗi và observability . . . 17
11.6. Health check và Service Discovery . . . 18
11.6.1. Health check — nền tảng cho tự động hóa . . . 18
11.7. User Activity Tracking — observability ở mức nghiệp vụ . . . 18
11.8. Chaos Engineering — kiểm tra khả năng phục hồi (resilience) chủ động . . . 19
11.8.1. Case study thực tế: Netflix Simian Army . . . 20
11.9. Case study: KBM . . . 21
11.9.1. Hiện trạng — đánh giá theo mô hình trưởng thành . . . 21
11.9.2. Phân tích theo bối cảnh nghiệp vụ . . . 22
11.9.3. Những cạm bẫy về observability . . . 23
11.10. Tổng kết . . . 24
11.11. Đọc thêm . . . 24
Tài liệu tham khảo . . . 25

CHƯƠNG 11

Quan sát hệ thống — Observability & Distributed Tracing

> Observability không chỉ là giám sát, mà là khả năng đặt câu hỏi mới về hành vi của hệ thống mà không phải triển khai mã mới.
> — Theo tinh thần Charity Majors

MỤC TIÊU HỌC TẬP

- Hiểu tại sao observability là **điều kiện tiên quyết** để vận hành microservices — không chỉ là thứ "có thì tốt"
- Phân biệt **observability và monitoring** và khi nào cần gì
- Nắm vững ba trụ cột: **Logs**, **Metrics**, **Traces** và cách chúng bổ trợ nhau
- Thiết kế chiến lược **log tập trung** (centralized logging) với log có cấu trúc (structured log), request ID và correlation ID
- Hiểu **truy vết phân tán** (distributed tracing) và các khái niệm trace, span, context propagation
- Nắm framework đo lường: **SLI/SLO/SLA** và chiến lược cảnh báo (alerting)
- Xây dựng **chiến lược xử lý lỗi** nhất quán giữa các service
- Phân tích các khoảng trống observability trong KBM và đề xuất lộ trình chuyển đổi

Phạm vi chương

Chương này tập trung vào nguyên tắc và thiết kế observability độc lập với hạ tầng triển khai cụ thể. Sau khi đọc Chương 12 (Triển khai & DevOps), người đọc sẽ thấy rõ hơn cách các công cụ observability được tích hợp vào CI/CD pipeline và môi trường chạy container.

Để vận hành hệ thống phân tán như KBM, nhóm phát triển cần chuyển từ tư duy giám sát thụ động (monitoring: hệ thống có đang hoạt động bình thường không?) sang khả năng quan sát chủ động (observability: tại sao hệ thống chạy chậm?) dựa trên ba trụ cột cốt lõi.

11.1. Ba trụ cột của observability

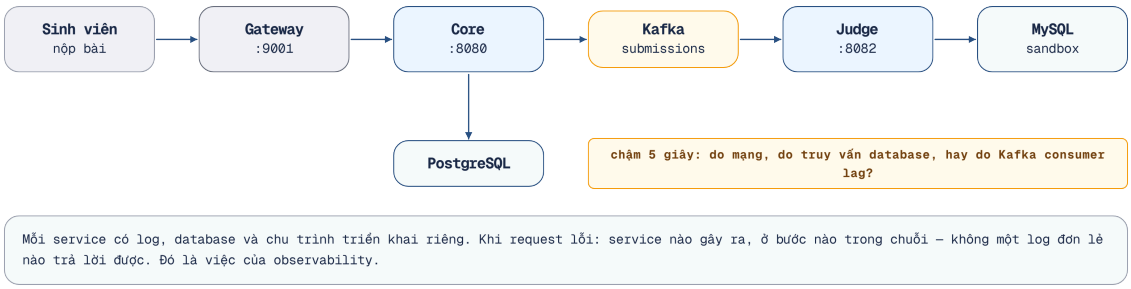
11.1.1. Vấn đề: "hệ thống lỗi nhưng không biết lỗi ở đâu"

Trong monolith, khi xảy ra lỗi: chỉ cần mở log file duy nhất, tìm ngoại lệ (exception) và đọc stack trace là có thể định vị được vấn đề. Toàn bộ request chạy trong cùng tiến trình, cùng log file, cùng database.

Trong kiến trúc microservices, một yêu cầu từ người dùng có thể đi qua nhiều dịch vụ, mỗi dịch vụ có nhật ký hệ thống, cơ sở dữ liệu và chu trình triển khai độc lập. Khi một yêu cầu gặp lỗi:

- Service nào gây lỗi? Gateway? Core? Judge? Auth?
- Lỗi xảy ra ở bước nào trong chuỗi gọi?
- Độ trễ 5 giây — do mạng, do truy vấn database, hay do Kafka consumer lag?

Hình 11.1 cho thấy một request đi qua hơn năm service — lỗi có thể nằm ở bất kỳ chặng nào.

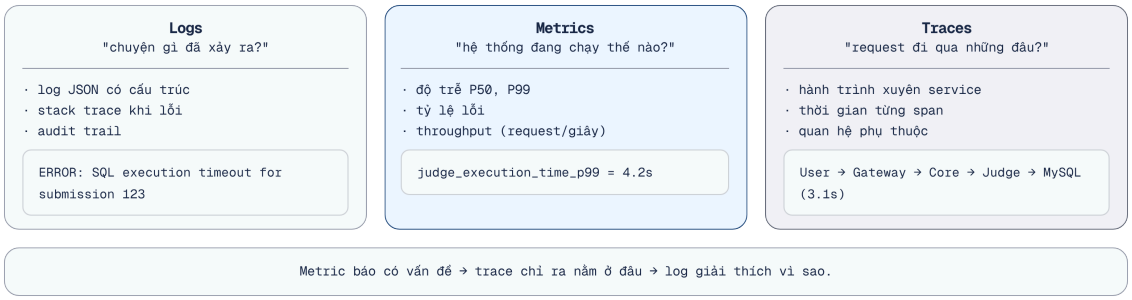


Hình 11.1: *Luồng request qua 5+ service — lỗi xảy ra ở đâu?*

Newman trong [4a, Ch.8] gọi đây là một thách thức lớn khi vận hành microservices: **sự phức tạp không nằm ở code, mà nằm ở tương tác giữa các service**. Trong monolith, lỗi thường có stack trace rõ ràng — một file, một method. Trong microservices, lỗi có thể xuất hiện ở dịch vụ B do dịch vụ A gửi dữ liệu sai, hoặc do message Kafka bị trễ — lúc này, không có stack trace nào ghi lại được nguyên nhân thật sự.

11.1.2. Ba trụ cột

Cộng đồng observability thường mô tả lĩnh vực này qua **ba trụ cột** (xem Sridharan, 2018); các kỹ thuật tương ứng có ở [2a, Ch.11] và [4a, Ch.8]:



Hình 11.2: *Ba trụ cột của observability — Logs, Metrics, Traces*

Hình 11.2 minh họa ba trụ cột ở mức tổng quan: mỗi trụ cột phục vụ một loại câu hỏi khác nhau khi gỡ lỗi hệ thống phân tán. Bảng dưới đây mô tả chi tiết vai trò, ví dụ cụ thể trong KBM, và các công cụ phổ biến cho từng trụ cột.

Bảng 11.1: *Vai trò của ba trụ cột observability*

Trụ cột	Trả lời câu hỏi	Ví dụ trong KBM	Công cụ phổ biến
Logs	"Chuyện gì đã xảy ra?"	`ERROR: SQL execution timeout for submission 123`	ELK Stack, Loki, Fluentd
Metrics	"Hệ thống đang hoạt động thế nào?"	`judge_execution_time_p99 = 4.2s`	Prometheus, Grafana, Micrometer
Traces	"Request đi qua những đâu?"	`User → Gateway → Core → Judge → MySQL (total: 3.1s)`	Jaeger, Grafana Tempo, Zipkin (dùng chuẩn OpenTelemetry OTLP)

M.E.L.T và hai nghĩa của Event

Trong công nghiệp, nhiều nền tảng giám sát (New Relic, Datadog) dùng thuật ngữ **M.E.L.T**, tách chữ *E* — *Events* thành yếu tố thứ tư. Cần phân biệt rõ: *Event* trong M.E.L.T là các **sự kiện vận hành/hạ tầng rời rạc** (triển khai phiên bản mới, khởi động lại container, sự kiện OOMKill, thay đổi cấu hình) giúp đối chiếu mốc thay đổi trên biểu đồ giám sát — khác với **Domain Event** mang ngữ nghĩa nghiệp vụ đi qua Message Broker (Kafka) đã học ở Chương 5–6.

Ba trụ cột **kết hợp với nhau:** metric cho biết *hệ thống đang có vấn đề* (thời gian phản hồi tăng), trace chỉ ra *vấn đề nằm ở đâu* (Judge Service chiếm 90% thời gian xử lý), log giải thích nguyên nhân *tại sao* (truy vấn SQL bị quá hạn do thiếu chỉ mục). Thiếu bất kỳ trụ cột nào, hai thành phần còn lại cũng mất đáng kể giá trị.

11.1.3. Observability và monitoring

Bảng 11.2 phân biệt hai khái niệm thường bị dùng lẫn này.

Bảng 11.2: *Phân biệt monitoring và observability*

	Monitoring	Observability
Mục đích	Phát hiện vấn đề đã biết	Khám phá vấn đề chưa biết
Cách tiếp cận	Đặt cảnh báo cho metric cụ thể	Khám phá dữ liệu để hiểu hành vi hệ thống
Ví dụ	"CPU > 80% → cảnh báo"	"Tại sao độ trễ tăng gấp 3 lúc 2 giờ chiều mỗi thứ Ba?"
Dữ liệu	Dashboard định sẵn	Logs + Metrics + Traces (kết hợp tùy biến theo nhu cầu)
Khi nào đủ	Hệ thống đơn giản, các kiểu lỗi (failure mode) biết trước	Hệ thống phân tán, các kiểu lỗi không dự đoán được

Monitoring trả lời: *"Có vấn đề không?"* Observability trả lời: *"Tại sao có vấn đề này, và vấn đề này khác gì so với lần trước?"* Với monolith, monitoring thường đủ. Với microservices — nơi các kiểu lỗi phức tạp và không dự đoán được — observability là yêu cầu bắt buộc.

Observability ≠ Monitoring

Monitoring cho biết *có* điều gì đó sai. Observability cho phép hỏi *vì sao*. Monitoring là một phần của observability: cần nhưng chưa đủ.

— Tổng hợp từ Charity Majors và cộng đồng observability

Camera giám sát và hệ thống quản trị học vụ

- **Monitoring**: giống như bác bảo vệ nhìn vào màn hình **camera giám sát** và thấy sinh viên đang ùn tắc trước thang máy nhà A. Bác chỉ biết: *hệ thống đang có vấn đề*.
- **Observability**: bác bảo vệ có hệ thống liên kết camera với lịch học và thẻ quẹt sinh viên. Tra cứu nhanh, bác phát hiện ra: *có 2 lớp học bù đột xuất được xếp trùng giờ ở cùng tầng 9*. Bác biết chính xác: *tại sao có vấn đề và nguyên nhân gốc rễ là gì*.

11.1.4. Mô hình trưởng thành observability (observability maturity model)

Không phải hệ thống nào cũng cần đầy đủ bộ công cụ observability ngay từ đầu. Mô hình trưởng thành giúp xác định *đang ở đâu* và *cần đến đâu*.

Bảng 11.3: *Mô hình trưởng thành observability*

Cấp độ	Mô tả	Đặc điểm	Phù hợp với
1 — Reactive	Gỡ lỗi khi người dùng báo lỗi	Console log, không có cảnh báo tự động, điều tra thủ công	Dự án cá nhân, bản mẫu thử nghiệm
2 — Basic monitoring	Dashboard cơ bản, cảnh báo thô	Health check, CPU/memory metric, log tập trung	Nhóm nhỏ, hệ thống ít dịch vụ
3 — Proactive	Phát hiện vấn đề trước khi người dùng gặp phải	Log có cấu trúc, request/correlation ID, metric tùy biến, cảnh báo	Nhóm quy mô vừa, nhiều dịch vụ
4 — Observability	Hỏi câu hỏi mới không cần triển khai code mới	Distributed tracing, SLI/SLO, dữ liệu cardinality cao	Nhóm quy mô lớn, hệ thống phức tạp
5 — Predictive	Dự đoán vấn đề trước khi xảy ra	Phát hiện bất thường dựa trên ML, hoạch định năng lực	Doanh nghiệp lớn, hệ thống trọng yếu

Nguồn: Tổng hợp từ Richardson [2a, Ch.11], Newman [4a, Ch.8], và Charity Majors, Liz Fong-Jones, George Miranda, Observability Engineering *(O'Reilly, 2022). Các cấp độ trưởng thành phản ánh kiến thức cộng đồng observability — không theo một framework độc quyền nào.*

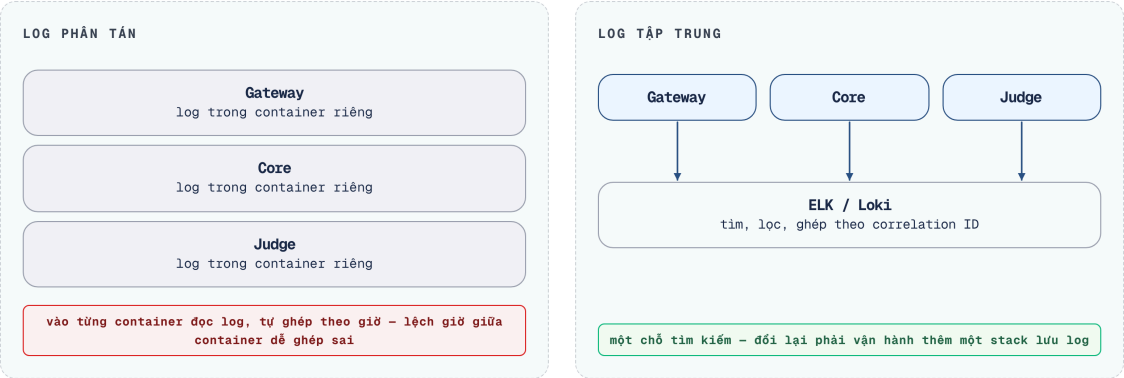
Với LMS — hệ thống giáo dục quy mô trung bình — mục tiêu hợp lý là **Cấp 3** (Proactive), với khả năng mở rộng lên Cấp 4 khi cần.

11.2. Logging — từ console đến gom log tập trung

Trong kiến trúc microservices, một request từ người dùng đi qua nhiều dịch vụ. Nếu log nằm rải rác trên nhiều máy chủ khác nhau, việc truy vết lỗi trở nên tốn kém và dễ bỏ sót.

11.2.1. Vấn đề: log phân tán, không có ngữ cảnh

Khi hệ thống có 7+ service chạy đồng thời, mỗi service ghi log vào container riêng. Để truy vết một yêu cầu thất bại, lập trình viên phải kết nối (SSH) vào từng container, đọc nhật ký và chắp vá thời gian — rất khó với các request đồng thời và lệch đồng hồ (clock skew) giữa các container.



Hình 11.3: *Log phân tán (gỡ lỗi thủ công) vs Log tập trung — tìm kiếm tại một nơi*

11.2.2. Log có cấu trúc — từ văn bản thô sang JSON

Bước đầu tiên: chuyển từ **log dạng văn bản** (text log) sang **log có cấu trúc** (JSON). Log dạng văn bản dễ đọc bằng mắt thường nhưng *rất khó phân tích tự động*.

Listing 11.1: *Ví dụ log có cấu trúc (JSON) theo chuẩn trường dữ liệu thống nhất*

```
// SAI: Text log — khó parse, khó filter, khó aggregate
2026-01-15 10:03:02 ERROR [kbm-core] - SQL execution failed for submission 8a3f
    user 12ab, Timeout after 30s

// ĐÚNG: Structured log (JSON) — dễ query, filter, aggregate
{"timestamp":"2026-01-15T10:03:02Z","severity":"ERROR","service.name":"kbm-core",
 "message":"SQL execution failed","submissionId":"8a3f","requestId":"req-8a3f12",
 "duration_ms":30000,"errorCode":"JUDGE_TIMEOUT",
 "error.type":"QueryTimeoutException",
 "traceId":"4bf92f3577b34da6a3ce929d0e0e4736","spanId":"00f067aa0ba902b7"}
```

Với log có cấu trúc, hệ thống có thể truy vấn: *"Tìm tất cả các lỗi trong 24h qua có* `error.type = QueryTimeoutException`*"* hoặc *"Thời gian xử lý trung bình của các lượt nộp bài theo giờ"* — điều không thể làm tin cậy với log dạng văn bản tự do.

Listing 11.2: *Khai báo Logback xuất JSON chứa ngữ cảnh truy vết và định danh (*`logback-spring.xml`*)*

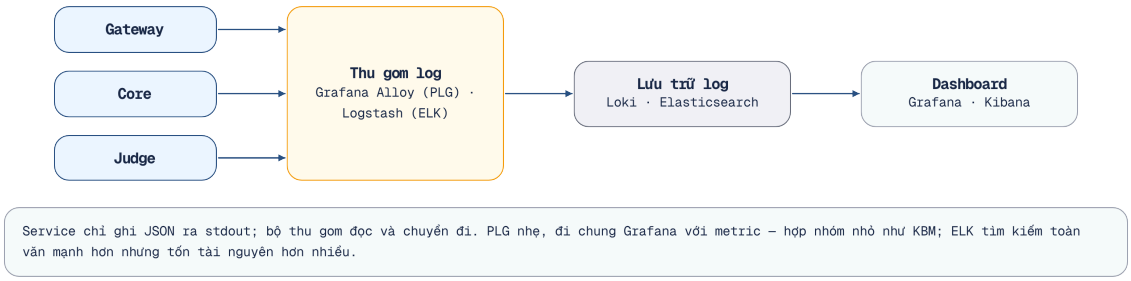
```
<configuration>
    <appender name="CONSOLE" class="ch.qos.logback.core.ConsoleAppender">
        <encoder class="net.logstash.logback.encoder.LogstashEncoder">
            <!— Tự động trích xuất ngữ cảnh từ MDC
                 (Micrometer Tracing + Request/Workflow Filter) —>
            <includeMdcKeyName>traceId</includeMdcKeyName>
            <includeMdcKeyName>spanId</includeMdcKeyName>
            <includeMdcKeyName>requestId</includeMdcKeyName>
            <includeMdcKeyName>correlationId</includeMdcKeyName>
        </encoder>
    </appender>
```

```
    <root level="INFO"><appender-ref ref="CONSOLE" /></root>
</configuration>
```

Từ Spring Boot 3.4 có thể bật log JSON bằng thuộc tính `logging.structured.format.console`, không cần thêm encoder.

11.2.3. Gom log tập trung (centralized log aggregation)

Khi log đã có cấu trúc, bước tiếp theo là **gom log tập trung** — thu gom log từ tất cả service về một nơi. Newman trong [4a, Ch.8] nhấn mạnh: *log tập trung là yêu cầu bắt buộc* cho microservices, không phải tùy chọn.



Hình 11.4: *Pipeline gom log tập trung — từ service đến nơi lưu trữ*

Để triển khai một pipeline thu thập log tập trung hiệu quả, các tổ chức thường đánh đổi giữa khả năng tìm kiếm toàn văn (full-text search) chuyên sâu và mức tiêu thụ tài nguyên phần cứng. Trong hệ sinh thái cloud-native, nổi bật nhất là hai stack tiêu chuẩn dưới đây:

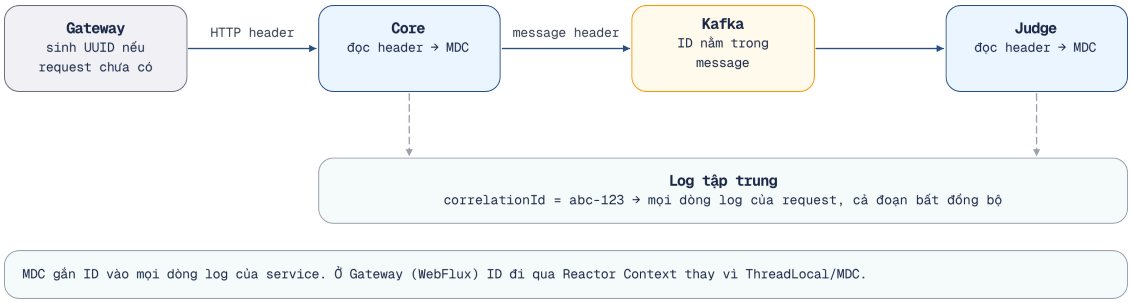
Bảng 11.4: *So sánh hai stack ELK và PLG*

Stack	Thành phần	Ưu điểm	Nhược điểm	Phù hợp
ELK	Elasticsearch + Logstash + Kibana	Trưởng thành, tìm kiếm toàn văn mạnh	Tốn tài nguyên (tối thiểu ~4 GB RAM)	Nhóm quy mô lớn, nhiều log
PLG	Promtail (đã hết vòng đời 03/2026, thay bằng Grafana Alloy) + Loki + Grafana	Nhẹ, tích hợp metric	Truy vấn kém linh hoạt hơn ELK	Nhóm nhỏ và vừa, KBM

11.2.4. Định danh xuyên suốt: Request ID và Correlation ID

Dù đã có hệ thống gom log tập trung, việc phân tích nguyên nhân gốc rễ (root cause analysis) vẫn rất khó nếu các dòng log của cùng một thao tác bị rời rạc. Newman trong [4a, Ch.8] mô tả **Correlation ID** là kỹ thuật nền tảng cho việc gỡ lỗi microservices: gắn định danh xuyên suốt từ điểm vào hệ thống qua các lời gọi HTTP và hàng đợi thông điệp. Như đã chuẩn hóa ở Chương 8 (§8.4), chúng ta phân biệt rõ vai trò của ba định danh để tránh dùng lẫn:

- `X-Request-ID` (trường log `requestId`): định danh từng HTTP request tại biên (Gateway kiểm tra định dạng hoặc sinh mới, chuyển tiếp xuống downstream và trả lại trong response header cho client).
- `correlationId`: định danh luồng nghiệp vụ đa bước (ví dụ một lượt nộp bài từ lúc HTTP `POST /submissions` tới các sự kiện Kafka và kết quả chấm), có thể bao phủ nhiều request và nhiều trace.
- `traceparent` / `traceId`: ngữ cảnh truy vết phân tán theo chuẩn W3C Trace Context (sẽ phân tích ở §11.3).



Hình 11.5: *Truyền định danh xuyên suốt request HTTP và luồng sự kiện Kafka*

Phiếu luân chuyển hồ sơ sinh viên

Hãy tưởng tượng một sinh viên nộp hồ sơ xin xét tốt nghiệp. Hồ sơ này được dán một mã vạch (`correlationId`) duy nhất cho toàn bộ quy trình, còn mỗi lần nộp bổ sung giấy tờ tại quầy một cửa có thêm số phiếu tiếp nhận (`X-Request-ID`). Bất kể hồ sơ đi qua Phòng Đào tạo (Core Service), Phòng Tài chính (Finance Service), hay Thư viện (Library Service), mọi phòng ban đều quét mã này và ghi vào sổ nhật ký. Nếu hồ sơ bị kẹt, Ban Giám hiệu chỉ cần tra mã là biết chính xác nó đang nằm ở khâu nào.

Cơ chế phối hợp đầu cuối:

1. **Gateway** kiểm tra định dạng `X-Request-ID` từ client (nếu thiếu hoặc sai định dạng thì sinh mới để chống log injection), gắn vào request chuyển tiếp và trả lại trong HTTP response header (Chương 8).
2. **Downstream service** đọc `X-Request-ID` ghi vào **MDC** (`requestId`), đồng thời gắn `correlationId` của luồng nghiệp vụ (chẳng hạn `submissionId`) vào MDC — logging framework tự động đính kèm vào mọi dòng log (Listing 11.2).
3. **Kafka producer/consumer** đặt `correlationId` (cùng `eventId`, `causationId` như đã học ở Chương 5) trong envelope của thông điệp để nối liền chặng bất đồng bộ.
4. **Khi gỡ lỗi**: tra `requestId` để xem một lượt gọi HTTP cụ thể, hoặc tra `correlationId` để xem toàn bộ vòng đời xử lý bài nộp qua nhiều dịch vụ.

Trên ứng dụng reactive như Spring Cloud Gateway (WebFlux), không ghi trực tiếp vào `ThreadLocal`/MDC trong thân filter vì một request có thể nhảy qua nhiều event-loop thread. Thay vào đó, filter ghi định danh đã làm sạch vào Reactor `Context` (Listing 11.3) và bật `Hooks.enableAutomaticContextPropagation()` (thư viện `micrometer-context-propagation`). Khóa `requestId` chỉ được đồng bộ sang MDC mỗi khi ghi log nếu ứng dụng đăng ký một `ThreadLocalAccessor` cho khóa MDC đó trong `ContextRegistry` (một lần, lúc khởi động):

Listing 11.3: *Chuẩn hóa `X-Request-ID` và truyền qua Reactor `Context` trong WebFlux*

```
@Component
public class RequestIdFilter implements GlobalFilter, Ordered {
    private static final String HEADER = "X-Request-ID";
    private static final Pattern SAFE_ID =
            Pattern.compile("^[A-Za-z0-9._-]{1,64}$");

    @Override
```

```
    public Mono<Void> filter(
            ServerWebExchange exchange, GatewayFilterChain chain) {
        String incoming = exchange.getRequest().getHeaders().getFirst(HEADER);
        String requestId = (incoming != null && SAFE_ID.matcher(incoming).matches())
            ? incoming
            : "req-" + UUID.randomUUID();

        // 1. Gắn vào request downstream và trả về response cho client
        ServerHttpRequest mutatedRequest = exchange.getRequest().mutate()
            .headers(h -> h.set(HEADER, requestId))
            .build();
        exchange.getResponse().getHeaders().set(HEADER, requestId);

        // 2. Lưu vào Reactor Context
        // (cầu nối sang MDC qua micrometer-context-propagation)
        return chain.filter(exchange.mutate().request(mutatedRequest).build())
            .contextWrite(Context.of("requestId", requestId));
    }

    @Override
    public int getOrder() { return Ordered.HIGHEST_PRECEDENCE; }
}
```

Mỗi request đều có một định danh xuyên suốt

"When the first call is made, you generate a GUID for the call. This is then passed along to all subsequent calls [...] and can be put into your logs in a structured way."

— Sam Newman, Building Microservices [4a, Ch.8]

11.3. Chi tiết về Distributed Tracing

Khi sinh viên phàn nàn hệ thống nộp bài quá chậm, thật khó để biết độ trễ nằm ở đâu nếu không có cơ chế theo dõi dấu vết. Distributed Tracing giải quyết bài toán này bằng cách vẽ lại chính xác đường đi của mỗi yêu cầu.

11.3.1. Vấn đề: request chậm — chậm ở đâu?

Các định danh tương quan (`requestId`, `correlationId`) giúp nối các dòng log lại — nhưng không cho biết **mỗi bước mất bao lâu**. Khi sinh viên nộp bài và thời gian phản hồi vọt lên 8 giây (so với khoảng 1 giây thường thấy ở truy vấn đơn giản), lập trình viên biết yêu cầu đã đi qua Gateway → Core → Judge → MySQL, nhưng không thể xác định thời gian dư thừa phát sinh ở bước nào nếu không dò timestamp trong log một cách thủ công.

11.3.2. Distributed Tracing — tự động đo thời gian

Distributed tracing ghi lại toàn bộ vòng đời của một request qua tất cả service, bao gồm thời gian mỗi bước (**span**):



Hình 11.6: Biểu đồ Gantt của một trace — thời gian của từng span

Trên một lượt chấm truy vấn đơn giản (~810 ms ở Hình 11.6), **Execute SQL on sandbox** chiếm 570 ms — tương đương ~70% tổng thời gian. Khi một bài nộp bất ngờ vọt lên 8 giây, cùng biểu đồ phân rã span này chỉ thẳng khâu bị kéo giãn (chẳng hạn hàng đợi sandbox hay khóa bảng MySQL) thay vì buộc lập trình viên phỏng đoán hoặc dò thủ công từng timestamp trong log.

11.3.3. Các khái niệm: Trace, Span, Context Propagation

Quy trình nhập học

- **Trace**: Là tổng thời gian từ lúc tân sinh viên bước vào trường đến lúc cầm thẻ sinh viên ra về (ví dụ: 120 phút).
- **Span**: Là thời gian sinh viên dừng lại ở từng trạm. Trạm "Đóng học phí" là một **span cha** (parent span, 30 phút).
- **Span con**: Bên trong trạm đóng học phí có 2 khâu nhỏ: "Xếp hàng chờ" (25 phút) và "Quẹt thẻ ngân hàng" (5 phút). Nhờ chia nhỏ thành span con (child span), chúng ta biết ngay "cổ chai" (bottleneck) không phải do hệ thống ngân hàng chậm, mà do thời gian chờ đợi xếp hàng quá lâu.

Ẩn dụ trên giúp hình dung trực quan quy trình tracing. Bảng dưới đây hệ thống hóa bốn khái niệm cốt lõi — Trace, Span, quan hệ cha-con, và Context propagation — cùng ví dụ cụ thể trong luồng nộp bài của KBM.

Bảng 11.5: Các khái niệm cơ bản trong Distributed Tracing

Khái niệm	Mô tả	Ví dụ
Trace	Toàn bộ vòng đời xử lý của một request	Nộp bài từ trình duyệt → trả điểm về
Span	Một bước trong trace (một đơn vị công việc)	"Core Service: save submission" (40 ms)
Cha-con (parent-child)	Span có thể chứa các span con	"Judge Service" span chứa "Execute SQL" và "Compare"
Context propagation	Truyền trace context qua ranh giới service	HTTP header `traceparent`, Kafka message header

11.3.4. Chiến lược lấy mẫu (Sampling)

Trên production, việc truy vết **100% request** sẽ tiêu tốn rất nhiều dung lượng lưu trữ và năng lực xử lý. Do đó, cần có chiến lược lấy mẫu (sampling strategy):

Bảng 11.6: Các chiến lược lấy mẫu cho tracing

Chiến lược	Mô tả	Khi nào dùng
Head-based	Quyết định lấy mẫu tại điểm vào (Gateway)	Đơn giản, dễ triển khai
Tail-based	Quyết định sau khi trace hoàn thành (giữ trace lỗi, bỏ trace thành công)	Ưu tiên giữ trace lỗi đã thu nhận, giảm dung lượng lưu trữ
Probabilistic (theo tỷ lệ)	Lấy mẫu N% request	Production lưu lượng cao (1–10%)
Adaptive	Tự động điều chỉnh tỷ lệ lấy mẫu theo lưu lượng	Doanh nghiệp lớn, tải biến động

Với KBM, cả trong chế độ thi tải vẫn ở mức hàng trăm lượt mỗi phút, nên có thể giữ 100% để không mất trace của bài nộp bị chậm; khi chi phí lưu trữ trở thành vấn đề thì chuyển sang tail-based — nhưng đây là đặc thù hệ thống nhỏ; với production có lưu lượng lớn, cần chọn chiến lược sampling phù hợp (probabilistic, tail-based) dựa trên tải thực tế và chi phí lưu trữ.

11.3.5. Tracing trong Spring Boot 3.x

Spring Boot 3.x tích hợp sẵn **Micrometer Tracing** — lớp trừu tượng (abstraction layer) cho distributed tracing. **OpenTelemetry (OTel)** hiện nay đã trở thành chuẩn mực công nghiệp thống nhất cho cả 3 trụ cột (Logs, Metrics, Traces), định nghĩa chuẩn truyền tải OTLP.

Để bật tracing và xuất dữ liệu ra OTel Collector, chúng ta chỉ cần khai báo các thư viện phụ thuộc (dependency) và cấu hình trong `application.yml` mà không phải sửa code logic:

Listing 11.4: Khai báo thư viện phụ thuộc cho tracing (`pom.xml`)

```
<!-- Spring Boot 3 Actuator -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
<!-- Micrometer OTel Bridge -->
<dependency>
    <groupId>io.micrometer</groupId>
    <artifactId>micrometer-tracing-bridge-otel</artifactId>
</dependency>
<!-- OTLP Exporter -->
<dependency>
    <groupId>io.opentelemetry</groupId>
    <artifactId>opentelemetry-exporter-otlp</artifactId>
</dependency>
```

Tuy nhiên, Distributed Tracing chỉ phát huy tối đa hiệu quả khi được kết hợp với Logs. Tracing giúp định vị "nút thắt" (bottleneck), còn Logs giải thích nguyên nhân gốc rễ (root cause).

Tracing Complements Logging

"Tracing xác định đường đi và thời gian xử lý của request, trong khi Logging giải thích nguyên nhân gây ra vấn đề. Sử dụng trace để nhận diện điểm nghẽn (bottleneck) và dùng log để hiểu nguyên nhân gốc rễ (root cause). Hai công cụ này bổ trợ cho nhau và không thể thay thế lẫn nhau."

— Tổng hợp từ Richardson [2a, Ch.11] và Newman [4a, Ch.8]

Sau phần thư viện phụ thuộc, việc còn lại là khai báo nơi xuất dữ liệu (OTLP exporter) và tỉ lệ lấy mẫu trong `application.yml` (Listing 11.5):

Listing 11.5: Cấu hình OTLP Exporter và Sampling (`application.yml`)

```yaml
management:
  endpoints:
    web:
      exposure:
        include: health,prometheus  # Expose metrics
  tracing:
    sampling:
      probability: 1.0  # Sample 100% (cho môi trường dev/staging)
  otlp:
    tracing:
      endpoint: http://otel-collector:4318/v1/traces
```

Với cấu hình này, Spring Boot **tự động** truyền trace context qua HTTP header theo chuẩn **W3C Trace Context** (header `traceparent` dạng `00-<traceId 32 hex>-<parentId 16 hex>-<trace-flags 2 hex>`, thay cho chuẩn B3 cũ). Để truyền qua Kafka, cần bật observation ở *cả hai phía*: `spring.kafka.template.observation-enabled=true` cho producer (`KafkaTemplate`) và `spring.kafka.listener.observation-enabled=true` cho consumer — thiếu phía producer thì trace sẽ đứt đoạn tại broker. Với `RestTemplate`, `RestClient` và `WebClient`, cần dựng từ builder do Spring Boot cung cấp thì context mới được truyền tự động; khi đó không cần viết code tracing thủ công cho phần lớn bài toán.

11.3.6. Kiến trúc triển khai Observability Pipeline

Phần nâng cao (có thể lướt qua)

Mục này dành cho hệ thống lớn. KBM ở quy mô hiện tại chưa cần Collector phân tầng.

Khi hệ thống lớn lên, ứng dụng không nên gửi trực tiếp dữ liệu đến Jaeger/Prometheus. Có 3 mô hình triển khai:

1. **Direct SDK**: Ứng dụng gửi trực tiếp dữ liệu đến hệ thống lưu trữ (tight coupling, dễ gây nghẽn mạng).
2. **Local Agent**: Ứng dụng gửi dữ liệu nội bộ tới agent chạy dưới dạng DaemonSet; agent tự quản lý bộ đệm và gom lô (batching).
3. **Centralized Collector**: Tất cả Agent gửi về một cụm Gateway Collector trung gian để che dữ liệu cá nhân (PII), lọc và định tuyến trước khi lưu trữ.

Nút thắt OTel Collector

Trong mô hình Centralized Collector, cụm Collector là điểm xử lý tập trung dễ trở thành nút thắt:

- **OOM (Out of Memory)**: Rất dễ xảy ra nếu dùng **Tail-based sampling**, vì Collector phải giữ tạm toàn bộ span của một trace trên RAM để đợi quyết định lưu trữ hay loại bỏ ở bước cuối cùng.
- **CPU Bound & Network Saturation**: Do phải phân tích (parse), lọc và biến đổi lượng dữ liệu log/trace lớn.

Biện pháp giảm thiểu: Triển khai kiến trúc phân tầng (Agent ở biên gom lô trước khi đẩy về cụm Gateway Collector), cấu hình bộ xử lý `memory_limiter` để chặn tràn RAM, dùng `loadbalancing`

`exporter` định tuyến mọi span cùng `traceId` về đúng một instance Collector khi lấy mẫu tail-based, và đặt Apache Kafka làm bộ đệm hấp thụ các đợt lưu lượng tăng đột biến. Ở tầng lưu trữ phía sau, áp thêm **chính sách vòng đời dữ liệu** (ví dụ chỉ giữ log nóng 7–14 ngày trên Loki rồi chuyển sang lưu trữ lạnh trên S3) để kiểm soát chi phí.

11.4. Metrics, SLI/SLO và chiến lược cảnh báo

Hệ thống cần các chỉ tiêu chất lượng rõ ràng để cảnh báo đội ngũ kỹ thuật kịp thời — trước khi sự cố kịp làm gián đoạn việc học của sinh viên.

11.4.1. Metrics — đo lường hành vi theo thời gian

Logs ghi lại *sự kiện đã xảy ra*. Metrics đo lường *xu hướng theo thời gian* — quan trọng cho việc **phát hiện sớm** (early detection) vấn đề trước khi người dùng báo lỗi.

Từ các observability pattern ở [2a, Ch.11], giáo trình tổng hợp bốn nhóm cần triển khai:

Bảng 11.7: *Bốn observability pattern cần triển khai*

Pattern	Mô tả	Ví dụ KBM
Health Check API	Endpoint `/health` cho hệ thống điều phối (orchestrator) biết dịch vụ đang hoạt động hay đã ngừng (up/down)	Eureka công bố trạng thái từ health check (khi bật tích hợp healthcheck, §11.6); LoadBalancer dựa vào đó để chọn instance
Application metrics	Metric ở mức nghiệp vụ	Tần suất nộp bài, thời gian chấm, tỷ lệ lỗi
Infrastructure metrics	Metric ở mức hệ thống	CPU, memory, DB connections, Kafka consumer lag
Audit logging	Ghi lại hành vi người dùng để đáp ứng yêu cầu tuân thủ (compliance)	Đăng nhập, thay đổi vai trò, truy cập dữ liệu

11.4.2. Bốn loại metric cơ bản của Prometheus

Trước khi áp dụng RED/USE, lập trình viên cần hiểu 4 kiểu dữ liệu đo lường:

1. **Counter**: Bộ đếm chỉ tăng lên (ví dụ: tổng số request). Tốt để tính Rate.
2. **Gauge**: Giá trị có thể tăng/giảm (ví dụ: mức dùng CPU, số kết nối đang mở). Tốt để tính Saturation.
3. **Histogram**: Nhóm dữ liệu vào các "bucket" (ví dụ: số request xong trong < 100 ms, < 500 ms). Nên dùng cho phân vị độ trễ (P95, P99) vì phía server tính lại và *gộp* được số liệu qua nhiều instance.
4. **Summary**: Ước lượng sẵn quantile ở phía client, theo sai số cấu hình, nhưng không gộp được giữa các replica, nên ít phù hợp với hệ phân tán nhiều bản sao.

Lưu ý rằng với độ trễ, các chỉ số vận hành luôn dùng **phân vị** (như P95, P99 tính từ Histogram) thay vì trung bình cộng (`average`). Trung bình cộng dễ che khuất đuôi trễ: nếu 99 sinh viên nhận kết quả trong 100 ms còn 1 sinh viên phải chờ 20 giây, giá trị trung bình chỉ khoảng ~300 ms — trông hoàn toàn bình thường dù 1% người dùng đang gặp trải nghiệm rất tệ.

⚠ Cardinality Explosion

Một sai lầm kinh điển của các nhóm mới làm quen với microservices là đưa `userId` hoặc `submissionId` vào làm nhãn (`label`) cho metric của Prometheus. Metric được thiết kế cho **Low Cardinality** (số lượng giá trị nhãn hữu hạn, ví dụ: `status=200`, `method=GET`). Nếu nhóm phát triển

gắn `userId` làm nhãn, cơ sở dữ liệu chuỗi thời gian (TSDB) sẽ sinh ra hàng triệu time-series khác nhau, ngốn hết bộ nhớ và làm sập hệ thống giám sát. Để tra cứu theo `userId`, hãy dùng Logs hoặc Traces.

Exemplars: cầu nối Metrics & Traces

Công nghệ hiện đại cho phép nhúng trực tiếp một `traceId` đại diện vào trong các bucket của Prometheus Histogram (gọi là **Exemplars**). Khi quan sát thấy một đỉnh trễ bất thường trên Grafana, kỹ sư có thể nhấp trực tiếp vào điểm dữ liệu đó để chuyển hướng đến Jaeger và phân tích chính xác trace gây ra sự chậm trễ.

Để quy chuẩn hóa việc đo lường trạng thái hệ thống, ngành công nghiệp thường sử dụng hai khuôn mẫu tiêu chuẩn: RED (cho dịch vụ) và USE (cho tài nguyên).

11.4.3. Hai khung đo lường: RED và USE

- **RED (dùng cho service phục vụ request)**: Bao gồm **R**ate, **E**rrors, và **D**uration. Ví dụ: tần suất nộp bài là 50 req/min, tỷ lệ lỗi là 2%, độ trễ P99 là 3.2 s.
- **USE (dùng cho tài nguyên hạ tầng)**: Bao gồm **U**tilization, **S**aturation, và **E**rrors. Ví dụ: CPU là 45%, DB connection pool là 8/10 active, Disk I/O errors là 0.

Kết hợp RED (cho service) + USE (cho tài nguyên) để giám sát cả hai khía cạnh của hệ thống. Metrics phát hiện "độ trễ nộp bài tăng" (RED), điều tra cho thấy "DB connection pool bị bão hòa" (USE) → nguyên nhân gốc rễ: rò rỉ kết nối (connection leak).

11.4.4. SLI/SLO/SLA — từ metric đến cam kết chất lượng

Metrics chỉ hữu ích khi biết **ngưỡng nào là chấp nhận được**. Khung SLI/SLO/SLA cung cấp ngôn ngữ chung:

Thư viện trường học

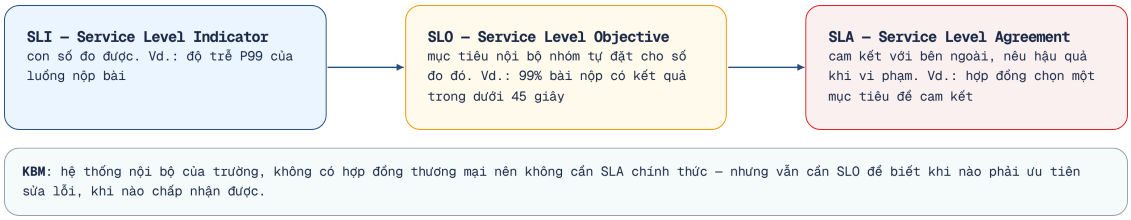
- **SLI (Chỉ số thực tế)**: Thời gian thực tế thủ thư dùng để xếp một cuốn sách trả lại lên kệ (ví dụ: 1.5 giờ).
- **SLO (Mục tiêu nội bộ)**: KPI mà giám đốc Thư viện đặt ra cho nhân viên: "99% sách trả lại phải được xếp lên kệ trong vòng 2 giờ".
- **SLA (Cam kết đền bù)**: Cam kết trong Sổ tay sinh viên: "Nếu sách người mượn trả chưa được hệ thống cập nhật trong ngày, nhà trường cam kết sẽ miễn trừ mọi khoản phạt trễ hạn tự động phát sinh".

Để đảm bảo chất lượng vận hành, hệ sinh thái đo lường được chia thành ba khái niệm chính:

- **SLI (Service Level Indicator)**: Là metric cụ thể được hệ thống đo lường trực tiếp để đánh giá chất lượng dịch vụ trong thực tế. Trong KBM, ví dụ tiêu biểu là đo "Độ trễ P99 của luồng nộp bài".
- **SLO (Service Level Objective)**: Là mục tiêu nội bộ mà đội ngũ kỹ thuật tự đặt ra đối với một SLI nhất định. Ví dụ: "99% bài nộp được trả kết quả chấm trong vòng 45 giây" (kịch bản chất lượng §1.5).

- **SLA (Service Level Agreement)**: Là những cam kết chính thức với khách hàng, có đi kèm với các hình phạt tài chính hoặc hậu quả pháp lý rõ ràng nếu hệ thống bị vi phạm. Ví dụ: "Hệ thống cam kết tính sẵn sàng (availability) đạt mức 99.5% tổng thời gian trong một học kỳ".

Hình dưới minh họa quan hệ giữa ba khái niệm: SLI là số đo để đối chiếu với mục tiêu SLO; SLA, nếu có, chọn các mục tiêu cam kết và nêu hậu quả khi vi phạm. Phần chênh lệch cho phép `100% - SLO` được gọi là **ngân sách lỗi** (error budget): chừng nào ngân sách lỗi trong kỳ đo còn dương, nhóm có thể ưu tiên phát hành tính năng mới; khi ngân sách lỗi cạn dần hoặc âm, mọi thay đổi rủi ro phải nhường chỗ cho việc gia cố độ tin cậy.



Hình 11.7: *Mối quan hệ giữa SLI, SLO và SLA*

Với KBM — hệ thống giáo dục nội bộ phục vụ sinh viên của trường — nhóm phát triển không cần SLA chính thức vì không có hợp đồng thương mại. Tuy nhiên, SLO nội bộ vẫn rất quan trọng: chúng đặt ngưỡng để nhóm phát triển biết khi nào cần ưu tiên khắc phục lỗi, khi nào chấp nhận được. Bảng dưới đề xuất năm SLO phù hợp với bối cảnh KBM:

Bảng 11.8: *SLO đề xuất cho KBM*

SLI	SLO	Lý giải
Thời gian tiếp nhận bài nộp P99	< 500 ms	Sinh viên cần biết ngay bài đã vào hàng đợi
Thời gian trả kết quả chấm P99	< 45 giây	Sinh viên chờ quá lâu dễ bấm nộp lặp hoặc bỏ dở phiên làm bài
Tính sẵn sàng trong ca thi	99.9% theo khung giờ ca thi (~5.4 giây gián đoạn / ca 90 phút)	Ca thi có thời gian giới hạn nghiêm ngặt; ngoài kỳ thi có thể áp mục tiêu 99.5% theo học kỳ
Tỷ lệ lượt chấm hoàn tất không lỗi hệ thống	≥ 99.9%	Tách bạch với tính đúng đắn tất định của bộ chấm (bất biến nghiệp vụ kiểm chứng bằng kiểm thử tự động); SLI vận hành đo tỷ lệ bài nộp không bị lỗi hạ tầng/sandbox
Tỷ lệ lỗi API (5xx)	< 1%	Tần suất lỗi máy chủ cao làm giảm trực tiếp độ tin cậy của hệ thống

11.4.5. Chiến lược cảnh báo

Tuy nhiên, nếu đội ngũ kỹ sư cấu hình cảnh báo quá nhạy cảm hoặc sai cách, họ sẽ nhanh chóng rơi vào trạng thái "mệt mỏi vì cảnh báo" (**alert fatigue**). Hiện tượng này xảy ra khi có quá nhiều cảnh báo giả (false positive) liên tục phát sinh, khiến các kỹ sư on-call dần sinh ra tâm lý chủ quan và bỏ qua ngay cả những sự cố thực sự nghiêm trọng. Để hạn chế tình trạng này, một cơ chế quan trọng cần áp dụng là **gom nhóm cảnh báo** — thay vì gửi 100 thông báo riêng lẻ khi một dịch vụ ngừng hoạt động (down) kéo theo hàng loạt lỗi liên quan, hệ thống sẽ gom chúng lại thành một thông báo duy nhất theo nguyên nhân gốc rễ.

Bảng 11.9: Chiến lược cảnh báo theo mức độ nghiêm trọng

Mức độ	Khi nào	Kênh	Hành động	Ví dụ
Nghiêm trọng (critical)	SLO bị vi phạm, ảnh hưởng đến người dùng ngay lập tức	PagerDuty, gọi điện	Kỹ sư trực hệ thống (on-call) phải xử lý ngay	Dịch vụ Judge ngừng hoạt động, nộp bài thất bại
Cảnh báo (warning)	Gần vi phạm SLO, cần chú ý	Slack, email	Xem xét trong giờ làm việc	Độ trễ P99 đạt 80% ngưỡng
Thông tin (info)	Bất thường nhưng không ảnh hưởng người dùng	Chỉ hiển thị trên dashboard	Xem xét sau hoặc phân tích định kỳ	Kafka consumer lag tăng nhẹ

Ba tầng cảnh báo ở trên giúp tránh tình trạng "mọi sự cố đều được coi là critical". Khi áp dụng thực tiễn, việc xây dựng tiêu chí cảnh báo cần gắn kết trực tiếp với trải nghiệm người dùng (thông qua SLO), thay vì dựa trên chỉ số kỹ thuật thuần túy. Chẳng hạn, nếu CPU tăng cao nhưng độ trễ vẫn trong giới hạn và người dùng không bị ảnh hưởng, thì không cần đánh thức kỹ sư on-call vào lúc 3 giờ sáng.

Cảnh báo theo triệu chứng, không theo nguyên nhân

"Cảnh báo khi SLO có nguy cơ bị vi phạm, không phải khi một metric thay đổi. CPU 90% nhưng độ trễ bình thường? Không cần cảnh báo. CPU 50% nhưng độ trễ vượt SLO? Cảnh báo ngay."

— Nguyên tắc SRE, dựa trên Google SRE book

11.5. Chiến lược xử lý lỗi — nhất quán giữa các service

Một hệ thống nhiều thành phần thường gặp tình trạng mỗi dịch vụ trả về một định dạng báo lỗi khác nhau. Để tránh gây nhầm lẫn cho ứng dụng frontend và sinh viên, hệ thống cần chuẩn hóa cách xử lý lỗi.

11.5.1. Vấn đề: mỗi service trả về định dạng lỗi khác nhau

Khi hệ thống bao gồm nhiều dịch vụ do các nhóm khác nhau phát triển, định dạng phản hồi lỗi (error response) rất dễ mất tính nhất quán. Dịch vụ A trả về lỗi trong trường `description`, Dịch vụ B lại sử dụng trường `message`, Gateway trả định dạng hoàn toàn khác. Ứng dụng khách (client/frontend) phải xử lý nhiều định dạng — mã nguồn phình phức tạp và trải nghiệm người dùng (UX) kém đi. Do đó cần thiết lập nguyên tắc chuẩn hóa.

11.5.2. Nguyên tắc thiết kế xử lý lỗi

Richardson trong [2a, Ch.11] và Newman trong [4a, Ch.8] đề xuất các nguyên tắc xử lý lỗi trong microservices:

1. Bộ xử lý ngoại lệ tập trung (centralized exception handler) — Mỗi service có một handler tập trung (trong Spring Boot: `@ControllerAdvice`) xử lý mọi ngoại lệ → chuyển thành định dạng chuẩn. Đặt trong thư viện dùng chung để mọi service cùng sử dụng.

2. Phân loại mã lỗi (error code taxonomy) — Phân loại lỗi theo namespace để client và hệ thống giám sát hiểu nhanh:

Bảng 11.10: Phân loại hệ thống mã lỗi (`errorCode` dạng chuỗi có namespace)

Namespace	Nhóm	Ví dụ `errorCode`
AUTH_*	Xác thực & phân quyền	AUTH_UNAUTHORIZED, AUTH_TOKEN_EXPIRED

Namespace	Nhóm	Ví dụ `errorCode`
NOT_FOUND_*	Không tìm thấy tài nguyên	NOT_FOUND_QUESTION, NOT_FOUND_CONTEST
BIZ_*	Vi phạm quy tắc nghiệp vụ	BIZ_CONTEST_NOT_STARTED, BIZ_SUBMIT_LIMIT
INFRA_*	Hạ tầng / phụ thuộc	JUDGE_UNAVAILABLE, KAFKA_SEND_FAILED (giữ tên phụ thuộc cho dễ đọc)
INTERNAL_*	Lỗi nội bộ	INTERNAL_UNEXPECTED

3. Problem Details (RFC 9457) — Mọi phản hồi lỗi dùng cùng cấu trúc Problem Details for HTTP APIs (RFC 9457), với các extension nội bộ KBM (`errorCode`, `traceId`):

Listing 11.6: Error response theo RFC 9457 Problem Details

```
HTTP/1.1 503 Service Unavailable
Content-Type: application/problem+json
X-Request-ID: req-a1b2c3

{
  "type": "urn:kbm:problem:judge-unavailable",
  "title": "Judge service unavailable",
  "status": 503,
  "detail": "Judge service did not respond within timeout.",
  "instance": "urn:kbm:problem-instance:req-a1b2c3",
  "errorCode": "JUDGE_UNAVAILABLE",
  "traceId": "0af7651916cd43dd8448eb211c80319c"
}
```

4. Không để lộ chi tiết nội bộ — Stack trace, câu truy vấn SQL, đường dẫn nội bộ không bao giờ xuất hiện trong phản hồi lỗi. Ghi log chi tiết ở phía server (kèm `traceId` và `requestId` để đối chiếu), chỉ trả thông báo chung cho client.

11.5.3. Mối liên hệ giữa xử lý lỗi và observability

Trong thực tế, các mã lỗi tập trung không chỉ phục vụ client — chúng là **nguồn dữ liệu quan trọng cho metric và cảnh báo**:

- Đếm lỗi theo mã → phát hiện xu hướng: `JUDGE_UNAVAILABLE` tăng đột biến → cảnh báo
- Gom lỗi theo nhóm → dashboard: lỗi xác thực, lỗi nghiệp vụ, lỗi hạ tầng
- Tỷ lệ lỗi theo service → SLI cho độ tin cậy

Định dạng lỗi không nhất quán trong KBM

KBM sử dụng `@ControllerAdvice` với `ErrorCode` enum trong thư viện dùng chung — đúng pattern. Tuy nhiên, **Gateway** (Spring Cloud Gateway dùng WebFlux) xử lý lỗi JWT *bên ngoài* `@ControllerAdvice`, vì lỗi ở tầng filter nằm ngoài phạm vi controller advice. Kết quả: Gateway trả phản hồi lỗi dạng `{ "message": "..." }` trong khi các service trả `{ "description": "..." }` — không theo RFC 9457 và tên trường khác nhau. Ứng dụng giao diện (frontend) phải xử lý đồng thời cả hai định dạng này.

Lộ trình chuyển đổi: (1) Tạo `GatewayExceptionHandler` riêng cho WebFlux, chuẩn hóa phản hồi theo `application/problem+json` (RFC 9457), (2) Đảm bảo mọi phản hồi lỗi — kể cả từ Gateway — dùng cùng schema Problem Details với `errorCode` (chuỗi) và `traceId`, (3) Loại bỏ các trường không thuộc hợp đồng truyền dữ liệu chuẩn (`timestamp`, `description`, `message`).

11.6. Health check và Service Discovery

Service Discovery và API Gateway chỉ có thể tự động ngắt định tuyến khỏi dịch vụ gặp sự cố khi có căn cứ: đó là vai trò của health check.

11.6.1. Health check — nền tảng cho tự động hóa

Richardson trong [2a, Ch.11] mô tả **Health Check API** là observability pattern cơ bản nhất: mỗi dịch vụ cần cung cấp (expose) một endpoint `/health` để các công cụ điều phối nhận biết trạng thái hoạt động của nó.

Health check phân loại theo mục đích:

Bảng 11.11: Ba loại health check

Loại	Câu hỏi trả lời	Ý nghĩa cho orchestrator	Ví dụ KBM
Liveness	"Tiến trình còn chạy?"	Khởi động lại container nếu tiến trình không phản hồi	JVM còn hoạt động, không bế tắc (deadlock)
Readiness	"Sẵn sàng nhận traffic?"	Ngừng chuyển traffic nếu dịch vụ chưa sẵn sàng	Đã kết nối cơ sở dữ liệu và Kafka
Startup	"Đã khởi tạo xong?"	Chờ quá trình khởi động hoàn tất mới kiểm tra liveness	Đã chạy migration schema, đã làm nóng cache

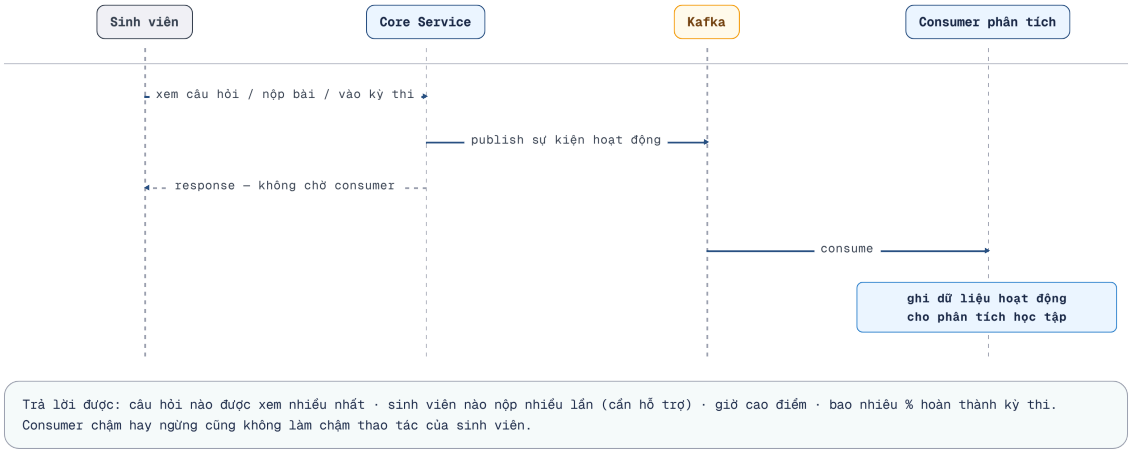
Trong kiến trúc dùng Eureka như KBM, health check chỉ thực sự tham gia vào định tuyến khi nối liền chuỗi trách nhiệm: instance Core Service báo `DOWN` qua endpoint health → **Eureka** (Service Registry) cập nhật trạng thái trong danh bạ (khi bật tích hợp healthcheck) → Spring Cloud LoadBalancer ở Gateway loại instance đó khỏi danh sách lựa chọn và chuyển traffic sang instance khỏe mạnh. DevOps Lab bổ sung health/readiness ở tầng runtime để router chỉ chuyển traffic đến workspace sẵn sàng. Còn việc *khởi động lại* instance hỏng thuộc về tầng điều phối/giám sát tiến trình (chính sách restart của Docker, Kubernetes) — kết hợp cả hai tầng mới tạo thành cơ chế **tự phục hồi** (self-healing) hoàn chỉnh không cần can thiệp thủ công.

> ⚠ **Eureka không tự suy ra trạng thái health của service**
>
> Một cấu hình dễ bỏ sót: mặc định Eureka client chỉ gửi *heartbeat* để báo tiến trình còn sống, nên một service vẫn được công bố `UP` dù endpoint health đã `DOWN` (chẳng hạn mất kết nối cơ sở dữ liệu) và tiếp tục nhận traffic lỗi. Để trạng thái từ Spring Boot Actuator được truyền lên registry, cấu hình client cần bật `eureka.client.healthcheck.enabled=true`. Khi bật cờ này, cũng cần cân nhắc kỹ mô hình kiểm tra sức khỏe: nếu endpoint health kiểm tra quá sâu vào các phụ thuộc bên ngoài, một sự cố nhất thời có thể khiến hàng loạt instance bị rút khỏi danh bạ cùng lúc (xem thêm phần phân biệt liveness và readiness ở cuối chương).

11.7. User Activity Tracking — observability ở mức nghiệp vụ

Ba trụ cột (log, metric, trace) giúp quan sát **hệ thống kỹ thuật**. Với một LMS như KBM, chúng ta còn cần quan sát **hành vi học tập**: sinh viên đang học gì, bài nào gây lúng túng nhất, khung giờ nào hoạt động sôi nổi nhất?

KBM triển khai **User Activity Tracking** qua đường ống Kafka: khi sinh viên xem câu hỏi, nộp bài hoặc tham gia kỳ thi, Core Service phát sự kiện theo dõi lên topic tương ứng. Consumer tiếp nhận và ghi nhận bất đồng bộ, không chặn (non-blocking) luồng xử lý chính của ứng dụng.



Hình 11.8: *User Activity Tracking pipeline qua Kafka*

Dữ liệu thu được từ pipeline này phục vụ nhiều mục đích, tóm tắt ở Bảng 11.12.

Bảng 11.12: *Ứng dụng của User Activity Tracking*

Dữ liệu	Câu hỏi trả lời được	Ứng dụng
Mẫu xem bài	“Câu hỏi nào được xem nhiều nhất?”	Đề xuất bài tập phù hợp
Mẫu nộp bài	“Sinh viên nào nộp bài nhiều lần?”	Phát hiện sinh viên cần hỗ trợ
Mẫu thời gian	“Giờ cao điểm là khi nào?”	Mở rộng hạ tầng phù hợp
Hành vi trong kỳ thi	“Bao nhiêu % sinh viên hoàn thành kỳ thi?”	Đánh giá hiệu quả giảng dạy

Đây là điểm giao giữa observability kỹ thuật và **phân tích đặc thù miền** (domain-specific analytics).

11.8. Chaos Engineering — kiểm tra khả năng phục hồi (resilience) chủ động

Observability cho phép **phát hiện** sự cố. Nhưng làm sao biết hệ thống sẽ phản ứng thế nào *trước khi* sự cố thật xảy ra? **Chaos Engineering** — phương pháp được Netflix phổ biến qua Chaos Monkey (2010) — trả lời câu hỏi này bằng cách **chủ động tiêm lỗi** (inject failures) vào môi trường staging/production và quan sát phản ứng. Đây là kỹ thuật ở giai đoạn trưởng thành cao (từ Cấp 4 trở lên); với quy mô như KBM, nhóm nên bắt đầu bằng các phép giả lập lỗi thủ công đơn giản trên staging.

Nguyên tắc (theo *Principles of Chaos Engineering* — principlesofchaos.org):

1. **Xác định trạng thái ổn định (steady state)** — hệ thống hoạt động bình thường trông như thế nào? (độ trễ < 200 ms, tỷ lệ lỗi < 0.1%, throughput > 100 req/s)
2. **Đặt giả thuyết** — “Nếu Judge Service ngừng hoạt động (down), bài nộp vẫn được xếp hàng và xử lý khi Judge khởi động lại”
3. **Tiêm lỗi** — Dừng đột ngột (kill) container Judge Service
4. **Quan sát** — Trạng thái ổn định có bị phá vỡ không? Hệ thống phục hồi trong bao lâu?
5. **Khắc phục điểm yếu** — Nếu trạng thái ổn định bị phá vỡ, cần sửa lỗi trên hệ thống (fix system) thay vì điều chỉnh lại kịch bản kiểm thử (fix test)

Diễn tập phòng cháy chữa cháy

Chaos Engineering giống như nhà trường cố tình bấm còi báo "Diễn tập phòng cháy chữa cháy" đột xuất ở Ký túc xá lúc 9 giờ tối. Mục đích là chủ động xem sinh viên có thoát hiểm kịp trong 5 phút không, cửa thoát hiểm có mở không — thay vì đợi đến khi có hỏa hoạn thật mới phát hiện ra cửa đã bị rỉ sét không mở được.

Tương tự như việc ban quản lý ký túc xá chuẩn bị nhiều kịch bản diễn tập khác nhau (chập điện, rò rỉ nước, hỏa hoạn), kỹ sư hệ thống cũng có các nhóm phép tiêm lỗi đa dạng để kiểm thử độ bền bỉ của nền tảng:

Bảng 11.13: *Phân loại kỹ thuật tiêm lỗi trong Chaos Engineering*

Loại lỗi được tiêm	Ví dụ	Công cụ
Lỗi tiến trình	Kill container/service	Chaos Monkey, `docker stop`
Lỗi mạng	Delay, packet loss, partition	Toxiproxy, Gremlin
Cạn tài nguyên	CPU 100%, disk full, memory leak	stress-ng, Gremlin
Lỗi phụ thuộc	Database down, Kafka unavailable	Litmus, thủ công

Bốn loại lỗi được tiêm trên phủ gần hết các miền lỗi (failure domain) trong hệ thống phân tán. Nhóm phát triển không cần chạy tất cả cùng lúc — nên bắt đầu từ ba thí nghiệm đơn giản nhất, vừa tầm với hạ tầng Docker Compose của KBM và đủ kiểm chứng các mẫu chịu lỗi đã thiết kế ở các chương trước:

- **Kill Judge Service**: Giả thuyết đặt ra là các bài nộp chế độ thi của sinh viên sẽ tự động được xếp hàng an toàn trong Kafka và xử lý tiếp khi service khởi động lại. Kết quả mong đợi là cơ chế lưu giữ của Kafka giữ toàn bộ thông điệp và Judge bắt kịp tiến độ ngay khi dịch vụ phục hồi.
- **Tiêm độ trễ mạng 6000 ms (vượt `readTimeout: 3500` và `timeoutDuration: 5s` của cấu hình Resilience4j ở §4.4.4) giữa Core↔Judge**: Giả thuyết đặt ra là khi luồng chấm đồng bộ phản hồi quá ngưỡng cho phép, cơ chế bảo vệ sẽ ngắt nhanh (fail-fast) để không làm treo nhóm luồng (thread pool) của Core Service. Kết quả mong đợi (khi đã triển khai timeout và circuit breaker ở Giai đoạn 3, Chương 10) là yêu cầu chấm quá hạn trả lỗi có kiểm soát thay vì kéo sập các API học vụ khác trên Core; chạy thí nghiệm này sớm hơn sẽ bộc lộ đúng khoảng trống đó.
- **PostgreSQL restart**: Giả thuyết là Core Service có khả năng tự động kết nối lại cơ sở dữ liệu. Kết quả mong đợi là connection pool (HikariCP) tự động khôi phục kết nối mà không yêu cầu khởi động lại ứng dụng.

Chaos Engineering từ staging

Không cần Chaos Monkey hay công cụ phức tạp cho những bước đầu tiên: chỉ với Docker CLI (`docker stop kbm-judge-sql` trên staging → quan sát log và metric → kiểm tra khả năng phục hồi), nhóm đã thực hành đúng tinh thần Chaos Engineering. Khi quy trình đã chín, mới tự động hóa thành bước kiểm thử trong pipeline CI/CD.

11.8.1. Case study thực tế: Netflix Simian Army

Netflix là người tiên phong của Chaos Engineering. Năm 2010, sau khi chuyển từ trung tâm dữ liệu sang AWS, Netflix tạo **Chaos Monkey** — công cụ tự động tắt ngẫu nhiên các EC2 instance trong production. Mục đích: nếu hệ thống chịu được các sự cố ngẫu nhiên hằng ngày, nó sẽ chịu được sự cố thật.

Chaos Monkey phát triển thành **Simian Army** — tập hợp các công cụ chuyên biệt:

Bảng 11.14: *Netflix Simian Army*

Công cụ	Chức năng	Bài học
Chaos Monkey	Kill ngẫu nhiên các instance	Các service phải stateless, tự khởi động lại được
Chaos Kong	Giả lập sự cố toàn bộ region	Cơ chế chuyển đổi dự phòng đa vùng (multi-region failover) phải hoạt động
Latency Monkey	Tiêm độ trễ mạng (network delay)	Timeout + circuit breaker phải được cấu hình đúng
Conformity Monkey	Kiểm tra instance tuân thủ các giải pháp khuyến nghị	Tự động hóa thay cho kiểm tra thủ công

Kết quả: Netflix duy trì độ sẵn sàng cao (mục tiêu ~99.99%) khi phục vụ hơn 200 triệu người đăng ký (2021). Casey Rosenthal — người dẫn dắt nhóm Chaos Engineering của Netflix, sau này đồng sáng lập Verica — cùng Nora Jones là đồng tác giả *Chaos Engineering: System Resiliency in Practice* (O'Reilly, 2020).

Bài học cho hệ thống nhỏ: LMS không cần Simian Army. Nhưng nguyên tắc cốt lõi áp dụng: *nếu đội ngũ phát triển e ngại việc vô hiệu hóa đột ngột (kill) một dịch vụ, đó là dấu hiệu hệ thống thiếu khả năng phục hồi*. Bắt đầu thủ công (`docker stop`), tiến tới tự động (CI/CD chaos stage).

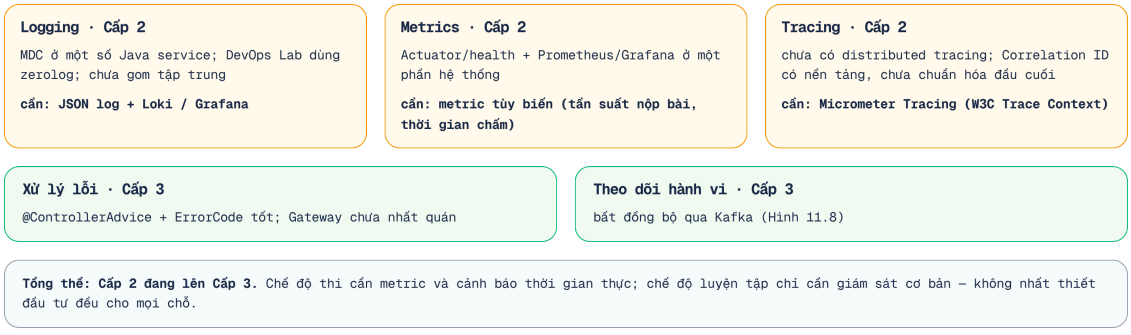
Nguồn: Netflix Technology Blog, "The Netflix Simian Army," 2011 (netflixtechblog.com). Casey Rosenthal et al., Chaos Engineering: System Resiliency in Practice, O'Reilly, 2020.

11.9. Case study: KBM

Phần này phân tích hiện trạng giám sát của KBM và vạch lộ trình nâng cấp tương ứng.

11.9.1. Hiện trạng — đánh giá theo mô hình trưởng thành

Hình 11.9 đánh giá hiện trạng observability của KBM theo từng thành phần.



Hình 11.9: *Đánh giá hiện trạng observability của KBM*

Đánh giá tổng thể: Cấp 2 đang tiến lên Cấp 3 — KBM đã có Actuator/health ở một số service và nền tảng Prometheus/Grafana cho một phần hệ thống, nhưng vẫn cần chuẩn hóa định danh tương quan, quy trình thu thập log và truy vết (tracing) xuyên suốt ranh giới các dịch vụ. Khi yêu cầu nộp bài phản hồi chậm hoặc thất bại, lập trình viên không nên phải mở nhật ký của từng container và ghép timestamp thủ công.

Bảng 11.15: Mức độ trưởng thành observability phân theo thành phần

Thành phần	Hiện trạng	Cấp độ trưởng thành
Logging	Các dịch vụ Java có log qua MDC ở một số nơi; DevOps Lab dùng zerolog; cần gom tập trung	Cấp 2
Metrics	Actuator/health + Prometheus/Grafana ở một phần hệ thống	Cấp 2
Tracing	Distributed tracing (W3C Trace Context) chưa triển khai; `X-Request-ID` và `correlationId` có nền tảng nhưng chưa chuẩn hóa đầu cuối	Cấp 2
Xử lý lỗi	`@ControllerAdvice` + `ErrorCode` tốt, Gateway chưa nhất quán	Cấp 3
Theo dõi hành vi người dùng	Bất đồng bộ qua Kafka — tốt	Cấp 3

11.9.2. Phân tích theo bối cảnh nghiệp vụ

KBM phục vụ sinh viên học SQL, mạng, DevOps và các hoạt động LMS. Điểm đặc biệt là hệ thống có nhiều chế độ hoạt động khác nhau — mỗi chế độ có mẫu lưu lượng, yêu cầu độ trễ và mức chịu lỗi riêng. Chiến lược observability không thể áp dụng một khuôn cho tất cả: chế độ thi cần cảnh báo thời gian thực, chế độ luyện tập chỉ cần giám sát cơ bản. Bảng dưới phân tích chi tiết từng chế độ:

Bảng 11.16: Các chế độ hoạt động chính của LMS

Chế độ	Đặc điểm	SLO cần thiết	Observability cần
Chế độ luyện tập	Sinh viên tự luyện tập, traffic thấp, không có hạn chót	Độ trễ < 10 s là được, downtime chấp nhận được	Giám sát cơ bản là đủ
Chế độ thi	khoảng 200 sinh viên nộp bài đồng thời, có hạn chót, bảng xếp hạng thời gian thực	Tiếp nhận P99 < 500 ms, trả kết quả P99 < 45 giây, tính sẵn sàng ca thi 99.9%	Metrics + cảnh báo bắt buộc
DevOps Lab runtime	Runtime của workspace/container/ mạng biến động	Readiness chính xác, cô lập lỗi	Grafana Alloy/Loki/Grafana + runtime metric

Trong chế độ thi, **không có observability = thiếu tầm nhìn hệ thống (flying blind)**:

- Judge Service quá tải → bài nộp dồn hàng đợi → bảng xếp hạng lỗi thời → sinh viên phàn nàn
- Không có metric tùy biến → không biết thời gian chấm tăng cho đến khi sinh viên báo
- Không có tracing → không biết nút thắt ở Judge hay ở Kafka consumer

Observability chưa chuẩn hóa đầu cuối

KBM đã có một số mảnh observability đúng hướng: Actuator/Prometheus cho các dịch vụ Java, `X-Request-ID`/MDC ở thư viện dùng chung, zerolog ở các dịch vụ Go, và bộ PLG (Promtail/Loki/Grafana) cho DevOps Lab. Tuy nhiên, hệ thống vẫn thiếu sự đồng nhất trên toàn chuỗi giao dịch.

Khoảng trống đáng kể hiện tại là **chuẩn hóa đầu cuối**: cùng một yêu cầu hoặc lô chấm bài đi qua Gateway → service → hàng đợi Kafka/DB → Judge/runtime cần giữ nguyên bộ định danh (`X-Request-ID` ở biên HTTP, `correlationId` cho luồng nghiệp vụ đa bước, `traceId` cho truy vết phân tán). Thiếu sự liền mạch này khiến việc gỡ rối bị đứt quãng giữa các tầng.

Để khắc phục, chúng ta cần một **lộ trình chuyển đổi** (tăng dần — mỗi giai đoạn đều mang lại giá trị ngay; mục tiêu là đạt đủ Cấp 3, riêng tracing là một phần của Cấp 4) như sau:

- **Giai đoạn 1 — Log có cấu trúc + Request ID** (chi phí triển khai thấp, tác động cao): Chuẩn hóa JSON logging cho các dịch vụ Java và tên trường zerolog cho Go; chuẩn hóa `X-Request-ID` lan truyền qua HTTP header (có kiểm tra đầu vào và gắn trả lại response) đồng thời truyền

`correlationId` trong message envelope trên Kafka. Giá trị nhận được ngay là khả năng tra cứu chéo dịch vụ chỉ bằng một định danh.

- **Giai đoạn 2 — Gom log tập trung** (chi phí triển khai trung bình): Mở rộng stack log đang chạy ở DevOps Lab (Loki + Grafana) sang LMS core, thu gom bằng Grafana Alloy vì Promtail đã hết vòng đời từ 03/2026. Việc cấu hình pipeline thu thập log tập trung sẽ chấm dứt ngay thói quen SSH vào từng container để đọc log thủ công.
- **Giai đoạn 3 — Distributed Tracing** (chi phí triển khai trung bình): Tích hợp Micrometer Tracing và OpenTelemetry vào mỗi dịch vụ, cùng với Jaeger làm backend, cho phép chúng ta nhìn rõ chính xác điểm nghẽn — liệu sự chậm trễ đến từ Judge Service, Kafka hay cơ sở dữ liệu.
- **Giai đoạn 4 — Metric nghiệp vụ + Cảnh báo** (chi phí triển khai trung bình): Mở rộng Prometheus/Grafana hiện có ra mọi service, thêm metric nghiệp vụ (tần suất nộp bài, thời gian chấm), từ đó thiết lập các cảnh báo dựa trên SLO thực tế như độ trễ nộp bài. Các cảnh báo sẽ được phân cấp hợp lý để gửi thông báo tức thời nếu Judge Service ngừng hoạt động, và phân mức cảnh báo độ trễ theo SLO cùng mức ảnh hưởng đến người dùng, không mặc định mọi bất thường độ trễ là cảnh báo nhẹ.

11.9.3. Những cạm bẫy về observability

Hệ thống giám sát thiết lập sai cách sẽ tự sinh nhiễu (noise), lãng phí tài nguyên và làm giảm độ nhạy bén của đội ngũ vận hành — độ chi tiết của dữ liệu phải được cân với chi phí. Những sai lầm phổ biến nhất:

1. **Log không cấu trúc, thiếu request/correlation ID** — Mỗi service ghi log dạng văn bản tự do riêng, không có ID xuyên suốt một request; khớp log bằng timestamp sai lệch ngay khi có xử lý đồng thời → debug như "mò kim đáy bể". *Phòng tránh*: log có cấu trúc (JSON) + `requestId` (HTTP ingress) và `correlationId` (luồng nghiệp vụ đa bước) lan truyền xuyên suốt mọi service.
2. **Logging quá nhiều hoặc quá ít** — Ghi toàn bộ request body làm lộ dữ liệu nhạy cảm và cạn kiệt lưu trữ; chỉ ghi log lỗi lại mất bối cảnh khi debug. *Phòng tránh*: INFO cho sự kiện nghiệp vụ, DEBUG cho chi tiết kỹ thuật; tuyệt đối không ghi mật khẩu hay PII.
3. **Cảnh báo theo nguyên nhân kỹ thuật thay vì theo SLO** — Cảnh báo cho mọi metric kỹ thuật rời rạc (CPU, một lỗi lẻ) sinh hàng chục thông báo mỗi ngày. Hậu quả: mệt mỏi vì cảnh báo (§11.4) — đội ngũ "miễn nhiễm" với cảnh báo, bỏ lỡ sự cố thật. *Phòng tránh*: cảnh báo dựa trên SLO và ngân sách lỗi (error budget) gắn với trải nghiệm người dùng, phân loại mức độ nghiêm trọng rõ ràng.
4. **Cardinality explosion trong metric** — Gắn giá trị biến thiên cao (`userId`, `submissionId`, `email`) làm nhãn cho Prometheus. Hậu quả: số time-series bùng nổ → quá tải hệ thống metric, tốn chi phí, truy vấn chậm. *Phòng tránh*: chỉ dùng nhãn có cardinality thấp (`status`, `method`, `service`); để chi tiết cho log/trace.
5. **Dùng tracing thay cho logging** — Tracing cho biết request đi đâu và tốn bao lâu, nhưng không giải thích *tại sao* thất bại. *Phòng tránh*: kết hợp cả hai — tracing cho cái nhìn tổng quan, logging làm rõ chi tiết.
6. **Liveness probe "đào" quá sâu** — Health check của liveness gọi tới DB hoặc downstream. Hậu quả: khi cơ sở dữ liệu xử lý chậm, Kubernetes nhận định sai rằng pod đã chết và kill tiến trình hàng loạt → sự cố lan rộng. *Phòng tránh*: liveness chỉ kiểm tra bản thân tiến trình (shallow); đưa kiểm tra phụ thuộc vào readiness.
7. **Bỏ qua observability vì "nhóm nhỏ"** — Ban đầu `docker logs` là đủ, nhưng khi hệ thống mở rộng, đọc log thủ công từ hàng chục container tiềm ẩn nhiều rủi ro vận hành. *Phòng tránh*:

bắt đầu ngay với log có cấu trúc + định danh tương quan — bước tốn ít công sức nhất nhưng giá trị nhất.

Các pattern trong chương có minh họa tương tác đi kèm tại companion web:

Interactive Demo

- **Truy vết phân tán (Distributed Tracing)**: distributed-tracing.html

11.10. Tổng kết

Observability là **điều kiện tiên quyết** để vận hành microservices — không phải tính năng bổ sung. Trong monolith, một tệp log và một trình gỡ lỗi (debugger) đủ để tìm lỗi. Trong microservices, yêu cầu đi qua nhiều service với các kiểu hỏng hóc khó dự đoán trước — cần khả năng *đặt câu hỏi mới* về hành vi hệ thống mà không phải triển khai bản mã mới.

Ba trụ cột — Logs, Metrics, Traces — bổ trợ đa chiều: metric phát hiện vấn đề (độ trễ tăng vọt), trace khoanh vùng vị trí (Judge Service), log chỉ ra nguyên nhân (SQL timeout). Mô hình trưởng thành giúp nhóm xác định *mức độ vừa sức*: với LMS như KBM, Cấp 3 (Proactive) là mục tiêu hợp lý.

Log có cấu trúc cùng bộ định danh (`requestId`, `correlationId`, `traceId`) là nền tảng có chi phí thấp nhưng tác động cao nhất. Trên nền đó, SLI/SLO và ngân sách lỗi chuyển trọng tâm từ "theo dõi chỉ số máy chủ" sang "bảo vệ trải nghiệm học tập", phản hồi lỗi chuẩn hóa theo RFC 9457 phục vụ đồng thời cả ứng dụng khách lẫn hệ thống cảnh báo, còn đường ống theo dõi hành vi qua Kafka mở rộng khả năng quan sát từ hạ tầng sang phân tích học tập.

Phân tích KBM cho thấy các mảnh ghép observability có sẵn cần được nối thành hợp đồng đầu cuối theo lộ trình tăng dần: log có cấu trúc + định danh tương quan → gom log tập trung → tracing → metric nghiệp vụ và cảnh báo theo SLO. Mỗi bước đều mang lại giá trị ngay lập tức.

Ở Chương 12, chúng ta sẽ chuyển sang **Triển khai và DevOps** — containerization, Docker Compose, pipeline CI/CD và cách triển khai hệ thống microservices hoàn chỉnh.

11.11. Đọc thêm

Sách tham khảo chính:

1. [2a] Chris Richardson, *Microservices Patterns* 1st Ed. — Ch.11: Developing Production-Ready Services — Health check API, Log aggregation, Distributed tracing, Application metrics
2. [4a] Sam Newman, *Building Microservices* — Ch.8: Monitoring — Logs, Metric tracking, Synthetic monitoring, Correlation IDs

Sách bổ trợ:

3. [5] Hugo Rocha, *Practical Event-Driven Microservices Architecture* — Distributed Tracing, Domain Segregation
4. [3] Ronnie Mitra & Irakli Nadareishvili, *Microservices: Up and Running* — Ch.9: Monitoring infrastructure
5. [4b] Sam Newman, *Monolith to Microservices* — Reporting, Monitoring and Troubleshooting during migration

Nguồn trực tuyến:

- OpenTelemetry official docs — opentelemetry.io

- Micrometer docs — micrometer.io (Spring Boot observability)
- Grafana Loki — grafana.com/oss/loki (lightweight log aggregation)
- Jaeger — jaegertracing.io (distributed tracing)
- Grafana Tempo — grafana.com/oss/tempo (tracing backend trong hệ sinh thái Grafana, một lựa chọn thay cho Jaeger)
- Brendan Gregg, "USE Method" — brendangregg.com/usemethod.html
- Tom Wilkie, "RED Method" — grafana.com/blog/2018/08/02/the-red-method-how-to-instrument-your-services/
- Google SRE Book, "Service Level Objectives" — sre.google/sre-book/service-level-objectives

Bài tập: 11-0, 11-1, 11-2 ở phần Bài tập & case study cuối sách.

Tài liệu tham khảo

[2a] Richardson, C. (2018). *Microservices Patterns: With Examples in Java*. Manning Publications.

[3] Mitra, R., & Nadareishvili, I. (2020). *Microservices: Up and Running: A Step-by-Step Guide to Building a Microservices Architecture*. O'Reilly Media.

[4a] Newman, S. (2015). *Building Microservices: Designing Fine-Grained Systems*. O'Reilly Media.

[4b] Newman, S. (2019). *Monolith to Microservices: Evolutionary Patterns to Transform Your Monolith*. O'Reilly Media.

[5] Rocha, H. F. O. (2022). *Practical Event-Driven Microservices Architecture: Building Sustainable and Highly Scalable Event-Driven Microservices*. Apress.